

Data Mining

Lab - 3

¥@\$# Kakadiya | 23010101121 |
18/6/2025

1) First, you need to read the titanic dataset from local disk and display first five records

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: df=pd.read_csv('../data/titanic.csv')
```

```
In [3]: df.head(5)
```

```
Out[3]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	

2) Identify Nominal, Ordinal, Binary and Numeric attributes from data sets and display all values.

```
In [4]: nominal=['Name','Ticket','Cabin']
ordinal=['Pclass','Embarked']
binary=['Survived','Sex']
numeric=['PassengerId','Fare','SibSp','Age','Parch']
print('Nominal:',nominal)
print('Ordinal:',ordinal)
print('Binary:',binary)
print('Numeric:',numeric)
```

```
Nominal: ['Name', 'Ticket', 'Cabin']
Ordinal: ['Pclass', 'Embarked']
Binary: ['Survived', 'Sex']
Numeric: ['PassengerId', 'Fare', 'SibSp', 'Age', 'Parch']
```

3) Identify symmetric and asymmetric binary attributes from data sets and display all values.

```
In [5]: symmetric=['Sex']
asymmetric=['Survived']
print('Symmetric:\n',df['Sex'].value_counts())
print('Asymmetric:\n',df['Survived'].value_counts())
```

```
Symmetric:
Sex
male      577
female    314
Name: count, dtype: int64
Asymmetric:
Survived
0        549
1        342
Name: count, dtype: int64
```

4) For each quantitative attribute, calculate its average, standard deviation, minimum, mode, range and maximum values.

```
In [6]: columns=['PassengerId','Survived','Pclass','Age','SibSp','Parch','Fare',];

for i in columns:
    print(i,': ')
    print('\tAverage: ', df[i].mean())
    print('\tStandard deviation: ', df[i].std())
    print('\tMinimum: ', df[i].min())
    print('\tMode: ', df[i].mode()[0])
    print('\tRange: ', df[i].max()-df[i].min())
    print('\tMax: ', df[i].max())
    print('-----')
```

```

PassengerId :
    Average: 446.0
    Standard deviation: 257.3538420152301
    Minimum: 1
    Mode: 1
    Range: 890
    Max: 891
-----
Survived :
    Average: 0.3838383838383838
    Standard deviation: 0.4865924542648585
    Minimum: 0
    Mode: 0
    Range: 1
    Max: 1
-----
Pclass :
    Average: 2.308641975308642
    Standard deviation: 0.8360712409770513
    Minimum: 1
    Mode: 3
    Range: 2
    Max: 3
-----
Age :
    Average: 29.69911764705882
    Standard deviation: 14.526497332334044
    Minimum: 0.42
    Mode: 24.0
    Range: 79.58
    Max: 80.0
-----
SibSp :
    Average: 0.5230078563411896
    Standard deviation: 1.1027434322934275
    Minimum: 0
    Mode: 0
    Range: 8
    Max: 8
-----
Parch :
    Average: 0.38159371492704824
    Standard deviation: 0.8060572211299559
    Minimum: 0
    Mode: 0
    Range: 6
    Max: 6
-----
Fare :
    Average: 32.204207968574636
    Standard deviation: 49.693428597180905
    Minimum: 0.0
    Mode: 8.05
    Range: 512.3292
    Max: 512.3292
-----

```

6) For the qualitative attribute (class), count the frequency for each of its distinct values.

```

In [7]: qualitative_attributes=['Pclass','Name','Ticket','Cabin','Embarked','Sex']
for i in qualitative_attributes:
    print(df[i].value_counts())
    print('-----')

```

```

Pclass
3      491
1      216
2      184
Name: count, dtype: int64
-----
Name
Braund, Mr. Owen Harris      1
Boulos, Mr. Hanna           1
Frolicher-Stehli, Mr. Maxmillian  1
Gilinski, Mr. Eliezer        1
Murdlin, Mr. Joseph          1
..
Kelly, Miss. Anna Katherine "Annie Kate"  1
McCoy, Mr. Bernard          1
Johnson, Mr. William Cahoon Jr  1
Keane, Miss. Nora A         1
Dooley, Mr. Patrick          1
Name: count, Length: 891, dtype: int64
-----
Ticket
347082      7
CA. 2343     7
1601         7
3101295      6
CA 2144      6
..
9234         1
19988        1
2693         1
PC 17612     1
370376       1
Name: count, Length: 681, dtype: int64
-----
Cabin
B96 B98      4
G6           4
C23 C25 C27  4
C22 C26      3
F33          3
..
E34          1
C7           1
C54          1
E36          1
C148         1
Name: count, Length: 147, dtype: int64
-----
Embarked
S      644
C      168
Q       77
Name: count, dtype: int64
-----
Sex
male      577
female    314
Name: count, dtype: int64
-----

```

7) It is also possible to display the summary for all the attributes simultaneously in a table using the `describe()` function. If an attribute is quantitative, it will display its mean, standard deviation and various quantiles (including minimum, median, and maximum) values. If an attribute is qualitative, it will display its number of unique values and the top (most frequent) values.

```
In [8]: print('Summary of numeric attributes:')
print(df.describe())
print()
print('-----')

print('Summary of all attributes:')
print(df.describe(include = 'all'))
print()
print('-----')

print('Summary of categorical attributes:')
print(df.describe(include = 'object'))
```

Summary of numeric attributes:

	PassengerId	Survived	Pclass	Age	SibSp \
count	891.000000	891.000000	891.000000	714.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008
std	257.353842	0.486592	0.836071	14.526497	1.102743
min	1.000000	0.000000	1.000000	0.420000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000

	Parch	Fare
count	891.000000	891.000000
mean	0.381594	32.204208
std	0.806057	49.693429
min	0.000000	0.000000
25%	0.000000	7.910400
50%	0.000000	14.454200
75%	0.000000	31.000000
max	6.000000	512.329200

Summary of all attributes:

	PassengerId	Survived	Pclass	Name	Sex \
count	891.000000	891.000000	891.000000	891	891
unique	NaN	NaN	NaN	891	2
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male
freq	NaN	NaN	NaN	1	577
mean	446.000000	0.383838	2.308642	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN

	Age	SibSp	Parch	Ticket	Fare	Cabin \
count	714.000000	891.000000	891.000000	891	891.000000	204
unique	NaN	NaN	NaN	681	NaN	147
top	NaN	NaN	NaN	347082	NaN	B96 B98
freq	NaN	NaN	NaN	7	NaN	4
mean	29.699118	0.523008	0.381594	NaN	32.204208	NaN
std	14.526497	1.102743	0.806057	NaN	49.693429	NaN
min	0.420000	0.000000	0.000000	NaN	0.000000	NaN
25%	20.125000	0.000000	0.000000	NaN	7.910400	NaN
50%	28.000000	0.000000	0.000000	NaN	14.454200	NaN
75%	38.000000	1.000000	0.000000	NaN	31.000000	NaN
max	80.000000	8.000000	6.000000	NaN	512.329200	NaN

	Embarked
count	889
unique	3
top	S
freq	644
mean	NaN
std	NaN
min	NaN
25%	NaN
50%	NaN
75%	NaN
max	NaN

Summary of categorical attributes:

	Name	Sex	Ticket	Cabin	Embarked
count	891	891	891	204	889
unique	891	2	681	147	3

top	Braund, Mr. Owen Harris	male	347082	B96	B98	S
freq		1	577	7	4	644

8) For multivariate statistics, you can compute the covariance and correlation between pairs of attributes.

```
In [9]: print('Covariance matrix:')
df.cov(numeric_only = True)
```

Covariance matrix:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	66231.000000	-0.626966	-7.561798	138.696504	-16.325843	-0.342697	161.883369
Survived	-0.626966	0.236772	-0.137703	-0.551296	-0.018954	0.032017	6.221787
Pclass	-7.561798	-0.137703	0.699015	-4.496004	0.076599	0.012429	-22.830196
Age	138.696504	-0.551296	-4.496004	211.019125	-4.163334	-2.344191	73.849030
SibSp	-16.325843	-0.018954	0.076599	-4.163334	1.216043	0.368739	8.748734
Parch	-0.342697	0.032017	0.012429	-2.344191	0.368739	0.649728	8.661052
Fare	161.883369	6.221787	-22.830196	73.849030	8.748734	8.661052	2469.436846

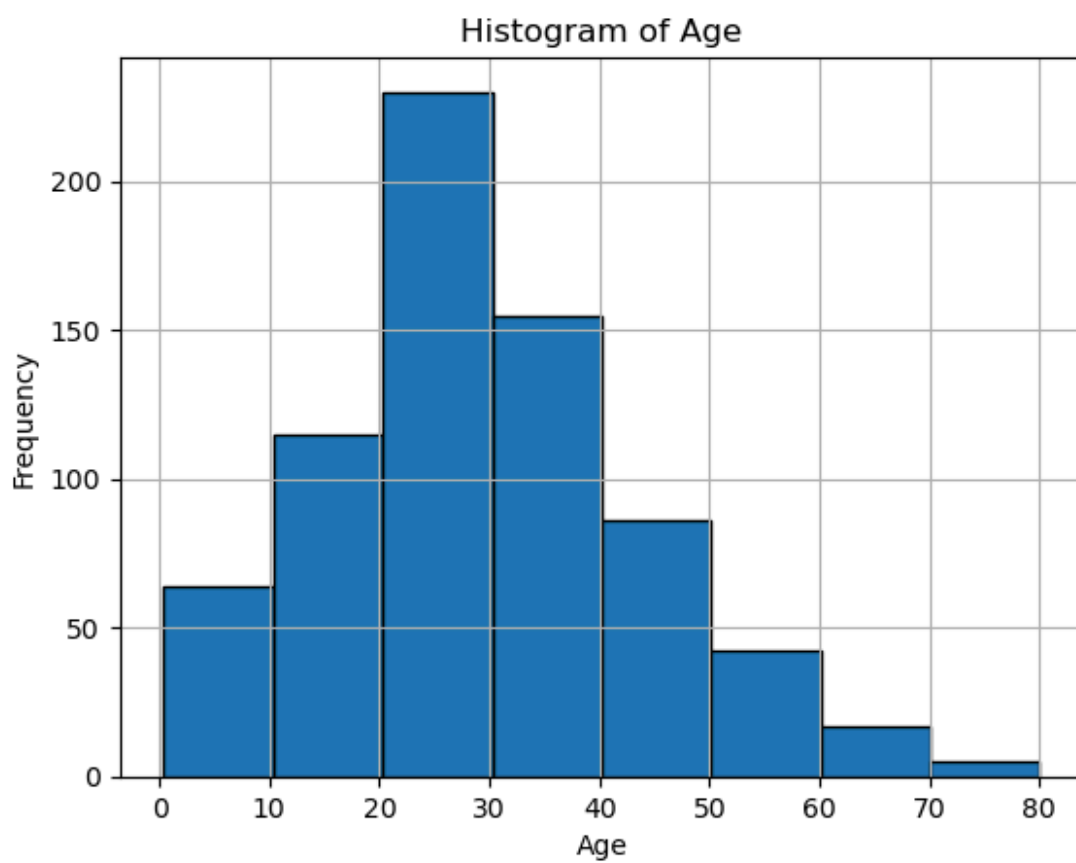
```
In [10]: print('Correlation matrix:')
df.corr(numeric_only = True)
```

Correlation matrix:

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

9) Display the histogram for Age attribute by discretizing it into 8 separate bins and counting the frequency for each bin.

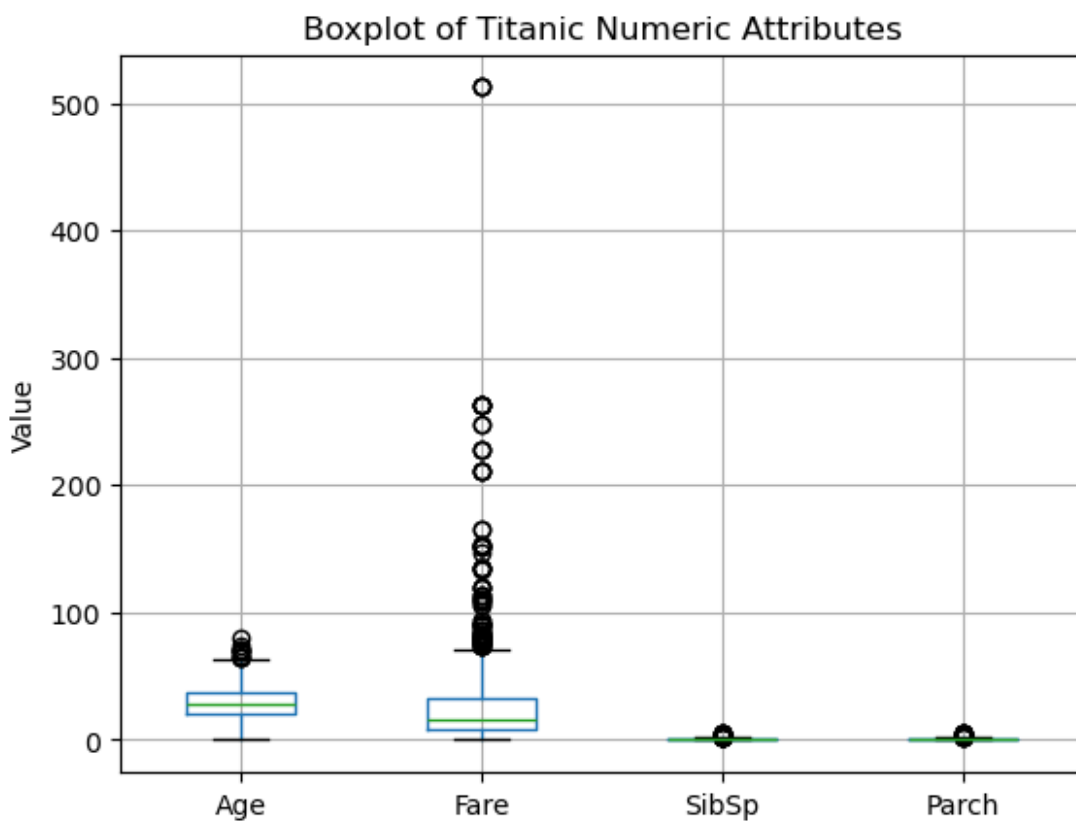
```
In [11]: df['Age'].dropna().hist(bins = 8, edgecolor = 'black', )
plt.title('Histogram of Age')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



10) A boxplot can also be used to show the distribution of values for each attribute.

```
In [12]: numeric = ['Age', 'Fare', 'SibSp', 'Parch']
data = df[numeric].dropna()

data.boxplot(column = numeric)
plt.title("Boxplot of Titanic Numeric Attributes")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```



11) Display scatter plot for any 5 pair of attributes , we can use a scatter plot to visualize their joint distribution.

```
In [13]: pairs = [ ('Fare', 'SibSp'), ('Fare', 'Parch'), ('Age', 'Fare'), ('Age', 'SibSp'), ('Age', 'Parch')

plt.figure(figsize = (15, 10))

for i, (x, y) in enumerate(pairs):
    plt.subplot(2, 3, i + 1)
    plt.scatter(df[x], df[y], alpha=0.7)
    plt.xlabel(x.capitalize())
    plt.ylabel(y.capitalize())
    plt.title(f"{x.capitalize()} vs {y.capitalize()}")
    plt.grid(True)
```

